

内核实现文档

ShellCore

北京科技大学

2026 年 8 月

目录

第 1 章 概述.....	4
1.1 ShellCore 整体结构	4
1.2 ShellCore 特性	4
1.3 小组成员分工	6
第 2 章 整体设计	7
2.1 项目结构	7
2.2 启动与初始化流程	9
2.3 异常处理	10
第 3 章 进程调度	11
3.1 任务组织结构	11
3.1.1 任务组织结构	11
3.1.2 系统调用 clone.....	13
3.1.3 系统调用 exec.....	13
3.1.4 内核工作线程	14
3.2 调度策略	14
3.2.1 任务状态与切换	14
3.2.2 调度策略	15
3.2.3 睡眠与阻塞/唤醒机制	16
3.2.4 futex 结构	17
3.3 睡眠相关	18

3.4 信号处理相关	18
3.4.1 信号处理	18
3.4.2 系统调用 wait	19
第 4 章 内存管理	20
4.1 地址空间	20
4.1.1 RISC-V	20
4.1.2 LoongArch	21
4.2 内存分配	23
4.2.1 内核堆分配	23
4.2.2 页帧分配与管理	23
4.3 高级特性与缺页处理	25
4.4 页缓存管理	25
4.5 Pin 页机制	27
第 5 章 文件系统	28
5.1 文件系统概述	28
5.2 文件描述符表	30
5.3 File Trait	30
5.4 vfs 层与 ext4 文件系统	31
5.4.1 vfs 抽象	31
5.4.2 ext4	31
5.5 文件页缓存使用	34
第 6 章 系统 IO 与板卡支持	36

6.1 IO 抽象	36
6.2 VisionFive2 支持	36
6.2.1 SD 卡支持	36
6.2.2 网卡支持	37
6.3 龙芯 2k1000 支持	37
6.3.1 板卡内存特性适配	37
6.3.2 SATA 磁盘	38
6.3.3 GMAC 以太网控制器	39
第 7 章 网络相关	41
7.1 底层网卡驱动与 smoltcp 适配.....	41
7.2 网络协议栈与接口管理	41
7.3 Socket 抽象.....	41
第 8 章 AI 使用声明	43
第 9 章 总结与展望	46
参考资料.....	47

第 1 章 概述

1.1 ShellCore 整体结构

ShellCore是一个基于“清华大学开源操作系统训练营rCore”开发的宏内核，使用rust语言编写。其支持RISC-V和Loongarch两种架构。下图是整体架构图：

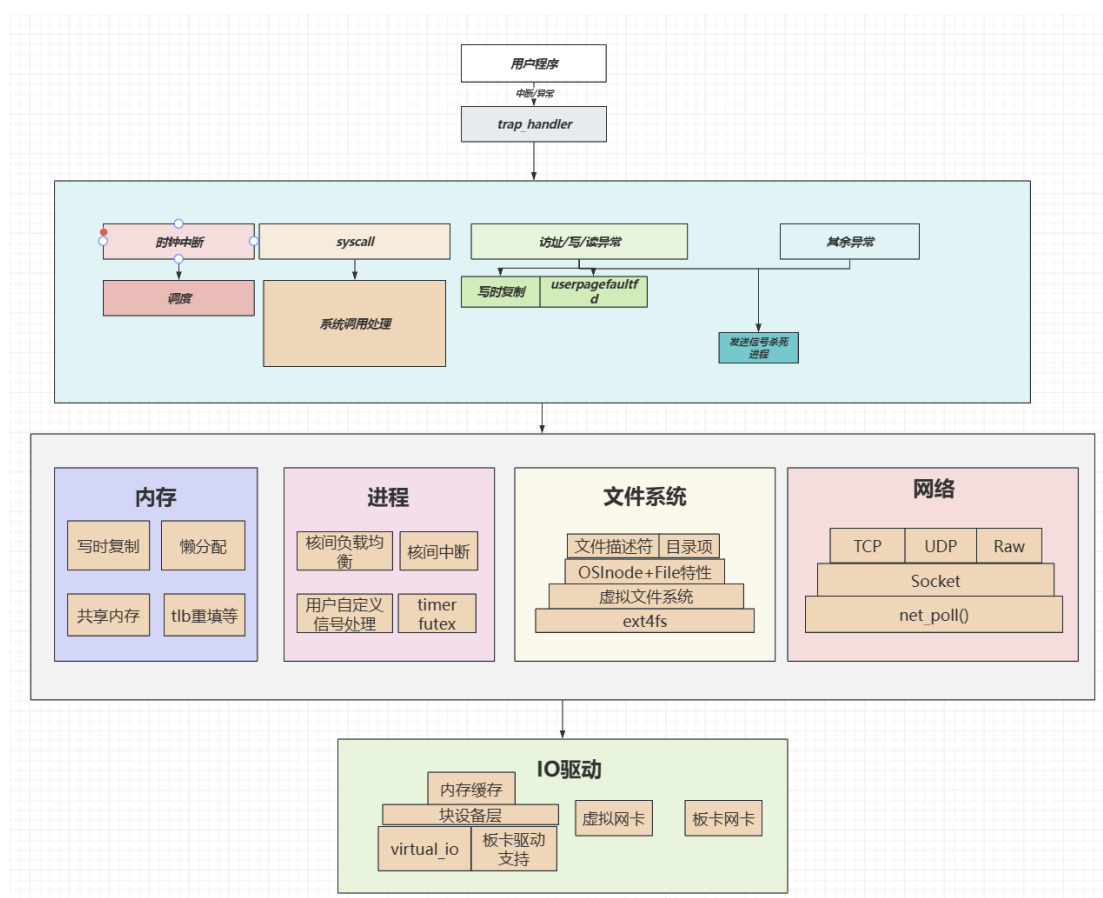


图 1.1 ShellCore 内核架构图

1.2 ShellCore 特性

ShellCore由rCore开发而来。截止至2026年8月13日，相较于rCore已经增加了如下支持：

基本结构： Loongarch/RISC-V低耦合兼容，SMP架构多核支持；

进程管理：阻塞唤醒机制，负载均衡与IPI唤醒，完整Linux语义信号处理，多队列CFS调度，完整的clone选项控制；

内存管理：pin页与延迟释放机制，大页支持，riscv用户引用内核页表，tlb远程同步刷新，写时复制，共享映射与文件映射，懒分配，exec懒加载；

文件系统：统一页缓存，文件与vfs抽象，ext4fs与tmpfs等虚拟文件系统支持，并发同步互斥机制；

IO设备：两架构四种板卡的块设备与网卡驱动，支持PCI扫描配置，与文件系统共用统一页缓存。

网络栈：

Linux兼容：160+系统调用，错误码规范返回，关键系统调用语义准确，POSIX语义睡眠唤醒，System V IPC等支持。

目前我们的仓库自2026年1月开始至今已有900+次提交，初赛赛题中，除ltp外其余的测试均已通过。决赛赛题两架构均能在限时内完成编译并qemu启动验证，拿到大部分分数。截止至8月16日，最近一次位于排行榜内的分数如下：

排名	题号	用户名	学校	子题数	子题1	子题2	子题3	子题4	总分
14	T2026181239910993	Unicus	哈尔滨工业大学 (深圳)	61	152.4	155.5	199.1	199.1	706.1000
15	T202610422999620	比特工匠队	山东大学 (青岛)	9	138.3	123.7	199.1	199.1	660.2000
16	T202610561999567	ya2yo	华南理工大学	63	150.0	104.1	199.1	199.1	652.3000
17	T202610008999575	ShellCore	北京科技大学	27	109.7	123.7	199.1	199.1	631.6000
18	T202610422999926	OuterSystems	山东大学 (青岛)	38	131.1	88.7	199.1	199.1	618.0000
19	T2026181239910974	Somber	哈尔滨工业大学 (深圳)	21	20.0	167.0	197.75	199.1	583.8500
20	T202610614999585	404NotFound	电子科技大学	65	8.0	180.0	195.75	199.1	582.8500
21	T2026105749910208	12345	华南师范大学	22	20.0	149.9	199.1	199.1	568.1000

图 1.2 2026 年 8 月 16 日决赛测试排行榜位置

1.3 小组成员分工

冯孟熙（队长）：负责早期smp架构设计；内存管理、文件系统的补充设计，统一页缓存的实现与优化；rCore（base）的riscv解耦与loongarch支持；2k1000板卡驱动适配。

唐博文：负责多核调度算法设计；进程管理如进程间通信、任务状态、任务的创建与回收等；写时复制等优化机制实现；文件系统架构设计与ext4兼容。

高铭均：负责网络栈的架构设计与底层驱动支持；星光二代开发板的板卡支持以及tmpfs等。

第 2 章 整体设计

2.1 项目结构

项目的目录结构如下：

1.	— Makefile	# 顶层构建入口：make test-rv / test-la / debug-* 等
2.	— os/	# 内核主体（RISC-V 与 LoongArch 双架构）
3.	— src/	
4.	— arch/	# 架构相关代码（riscv/、la/）
5.	— drivers/	# 磁盘、网络、串口等设备驱动
6.	— ext4fs/	# ext4 文件系统实现
7.	— fs/	# 文件系统抽象、页缓存与块缓存
8.	— mm/	# 内存管理：地址空间、页表、RCU 等
9.	— process/	# 线程组 / 进程模型与调度器
10.	— sync/	# 锁、等待队列等同步原语
11.	— ipc/	# 进程间通信
12.	— net/	# 网络协议栈
13.	— syscall/	# 系统调用层
14.	— auth/	# 权限与用户身份
15.	— ...	
16.	— build.rs	
17.	— Cargo.toml	
18.	— user/	# 用户态程序与初始化程序
19.	— src/bin/	# initproc 等二进制
20.	— user-la/	# LoongArch 用户程序
21.	— user-rv/	# RISC-V 用户程序
22.	— boot/	# uImage 打包脚本、busybox 等启动相关内容
23.	— toolkits/	# 开发板烧写、测评镜像制作等工具
24.	— benchmarks/	# 性能基准与对比测试（mmbench、sysbench、iozone 等）
25.	— docs/	# 参赛文档、日志与测试结果
26.	— ref/	# 参考资料（Linux 6.18 源码、龙芯手册）
27.	— inst	# 板卡调试命令备忘
28.	— ...	

整体采用去耦合设计，两种架构的差异/底层实现分别位于 `os/src/arch/riscv` 和 `os/src/arch/la`，通过 `#[cfg()]` 语句块条件编译，暴露统一接口，从而使上层抽象与架构解耦（只需单次实现）：

```
1. //! os/arch/mod.rs
2. #![allow(dead_code)]
3.
4. #[cfg(target_arch = "riscv64")]
5. pub mod riscv;
6. #[cfg(target_arch = "loongarch64")]
7. pub mod la;
8.
9. #[cfg(target_arch = "riscv64")]
10. pub use riscv::*;
11. #[cfg(target_arch = "loongarch64")]
12. pub use la::*;
```

位于 `user` 的初始化程序将会在 `make` 时被编译，并按情况拷贝到 `la` 或 `riscv`，详见 `Makefile`。如果不先运行 `make` 准备初始化程序，您的语法分析器可能会报错找不到 `initproc.elf`，但这并不是真正的语法错误。

内核编译时会将其链接到内核 `elf` 的 `.data` 段，并在启动时作为初始化进程：

```
1. #[repr(C)]
2. struct InitProcData<T: ?Sized> {
3.     pub _align: [u64; 0],
4.     pub bytes: T,
5. }
6.
7. #[link_section = ".data"]
8. #[cfg(target_arch = "riscv64")]
9. static INITPROC_DATA: &'static InitProcData<[u8]> = &InitProcData {
10.     _align: [],
11.     #[cfg(initproc = "default")]
12.     bytes: *include_bytes!("../arch/riscv/initproc"),
13.     #[cfg(initproc = "sh")]
```

```
14. bytes: *include_bytes!( "../arch/riscv/initproc_sh"),
15. #[cfg(initproc = "ltp")]
16. bytes: *include_bytes!( "../arch/riscv/initproc_ltp"),
17. #[cfg(initproc = "mmtest")]
18. bytes: *include_bytes!( "../arch/riscv/initproc_mmtest"),
19. #[cfg(initproc = "cargotest")]
20. bytes: *include_bytes!( "../arch/riscv/initproc_cargotest"),
21. #[cfg(initproc = "ctidtest")]
22. bytes: *include_bytes!( "../arch/riscv/initproc_ctidtest"),
23. #[cfg(initproc = "uptime")]
24. bytes: *include_bytes!( "../arch/riscv/initproc_uptime"),
25. #[cfg(initproc = "waitbench")]
26. bytes: *include_bytes!( "../arch/riscv/initproc_waitbench"),
27. #[cfg(initproc = "mmbench")]
28. bytes: *include_bytes!( "../arch/riscv/initproc_mmbench"),
29. };
```

2.2 启动与初始化流程

启动流程如下：

1. 启动核（主核）执行 `_start`，按照规则设置 `sp` 到自己的启动栈顶，并保存 `hart_id` 到 `tp` 寄存器，随后跳转到 `rust_main()`；
2. 主核继续跳转到 `main_init()`，完成内存等的初始化，随即唤醒非启动核（副核）；
3. 随后，副核也执行上述流程，但只执行本核特有初始化；
4. 初始化完成后，多核分别调用 `run_tasks()`，即各自的调度循环；
5. 内核运行的第一个用户程序将是初始化进程。

特别地：

- **Loongarch**：使用映射窗口，内核态总是不使用页表。启动前由于物理地址位数低，内核 `elf` 的高位被截断，在物理地址启动时设置映射窗口随后打开虚拟地址，此时地址被扩展到 64 位，内核 `elf` 的高位地址恢复，使内核落在映射窗口中；

- **riscv**: 启动阶段会用临时页表将内核同时映射到高半地址空间和低半地址空间;
- **多核唤醒**: RV使用了sbi_call, LA参考了RocketOS 的IPC的多核唤醒; 尽管在称谓上有主核与副核的区分, 但只在启动流程上有区别, 初始化完成后二者的地位是完全等同的。

2.3 异常处理

进入trap_handler()前先保存上下文, 这部分参考rCore, 但针对多核和龙芯有一些修改。完成保存进入正式异常处理阶段后, 根据异常寄存器分配异常:

- **系统调用**: 处理系统调用;
- **缺页异常**: 处理COW与懒分配懒加载, 均处理失败则报段错误;
- **时间片轮转**: 保存任务上下文切换到run_tasks()调度循环。

异常处理完毕后进行信号处理 (可能前往用户处理函数), 最后恢复上下文返回用户态。

第 3 章 进程调度

3.1 任务组织结构

3.1.1 任务组织结构

初赛的PCB嵌套TCB的旧实现下，同一进程下的不同线程在系统调用中会争抢同一把PCB锁，且不太能适配Linux clone语义。因此我们最终将任务控制块改为了类似Linux以线程（任务）为单位的单层TaskStruct结构：

```

1. pub struct TaskStructInner {
2.     pub on_main_hart: bool, // 是否在主核上运行
3.     // 命名空间
4.     pub nsproxy: Arc<NsProxy>,
5.     /* 0. 上下文 */
6.     pub thread: ThreadStruct, // 线程上下文，保存寄存器等信息
7.     /* 1. 进程标识信息 */
8.     pub group_leader: Weak<TaskStruct>, // 线程组领头进程
9.     pub kernel_stack: KernelStack,
10.    /* 2. 进程亲缘关系 */
11.    pub real_parent: Weak<TaskStruct>, // 实际创建当前进程的父进程
12.    pub parent: Weak<TaskStruct>,      // 接收 SIGCHLD 信号的父进程
13.    pub children: Vec<Arc<TaskStruct>>, // 子进程链表头
14.    pub pgid: usize, // 进程组 id
15.    pub sid: usize,  // 会话 id
16.    /* 3. 进程状态 */
17.    pub state: TaskStatus, // 进程运行状态
18.    /// 阻塞切换期间收到唤醒请求（由唤醒者置位，调度器完成切换后消费）
19.    pub wake_pending: bool,
20.    /// 在 BlockSaving 期间发起唤醒的CPU；提交阻塞切换后据此重新选择目标CPU。
21.    pub wake_source_cpu: Option<usize>,
22.    pub exit_state: i64, // 进程退出状态
23.    pub exit_code: i32,  // 进程退出码
24.    pub exit_signal: i32, // 进程退出信号
25.    pub flags: u32,      // 进程特性标志
26.    pub errno: i32,      // 进程错误码
27.    /* 4. 进程调度相关 */
28.    /// 用户设置的调度策略，例如 SCHED_OTHER、
    SCHED_RR 或 SCHED_DEADLINE。

```

```

29. pub sched_policy: isize,
30. /// 用户可见的实时优先级；普通策略为0，实时策略为1..=99。
31. pub sched_priority: i32,
32. /// 调度器当前使用的动态优先级。
33. pub prio: i32,
34. /// 由 nice 或实时参数计算出的静态优先级。
35. pub static_prio: i32,
36. /// 不考虑临时优先级继承时的正常优先级。
37. pub normal_prio: i32,
38. /// 普通公平调度实体。
39. pub se: SchedEntity,
40. /// FIFO/RR 实时调度实体。
41. pub rt: SchedRtEntity,
42. /// Deadline/CBS 调度实体。
43. pub dl: SchedDlEntity,
44. /* 5. 内存管理相关 */
45. pub mm: Option<Arc<MemorySet>>, // 用户进程内存描述符
46. // pub active_mm: *mut mm_struct, // 上下文切换使用的活动 mm
47. /* 6. 文件系统与文件描述符 */
48. pub fs: Arc<MPSafeCell<FsStruct>>, // 进程当前目录、根目录信息
49. pub files: Arc<MPSafeCell<FileDescriptorTable>>, // 进程打开的文件描述符表
50. /// 当前进程映像对应的规范绝对路径，用于 /proc/<pid>/exe。
51. pub exe_path: String,
52. /* 7. 信号处理相关 */
53. pub signal: Arc<MPSafeCell<Signal>>, // 信号处理相关信息
54. pub exec_update_lock: Arc<AtomicBool>, // 线程组 exec 互斥锁
55. pub signal_hand: Arc<MPSafeCell<SigHand>>, // 信号处理函数相关信息
56. pub blocked: SignalFlags, // 当前阻塞（不允许接收）的信号集
57. pub pending: Sigpending, // 当前挂起（收到但还未处理）的信号集
58. pub signal_interrupted: bool,
59. pub sigsuspend_saved_mask: Option<SignalFlags>,
60. pub signal_mask_backup: Vec<SignalFlags>,
61. pub trap_ctx_backup: Vec<TrapContext>,
62. pub signal_user_context_backup: Vec<usize>,
63. pub signal_alt_stack: SignalAltStack,
64. pub term_signal: Option<i32>,
65. pub frozen: bool,
66. /* 8. gid uid 等 */
67. pub cred: Arc<MPSafeCell<Cred>>, // 进程的凭证信息
68. pub real_cred: Arc<MPSafeCell<Cred>>, // 进程的真实凭证信息
69. /* 9. 时间相关 */
70. pub start_time: u64, // 进程启动时间
71. pub start_boottime: u64, // 进程启动时间的低位
72. /// 进程的 oom_score_adj（范围 -1000..=1000，LTP 兼容）

```

```
73. pub oom_score_adj: i32,
74. /* 10. CPU 调度 */
75. pub on_cpu: bool,
76. pub on_rq: bool,
77. pub cpu: usize,
78. /// 允许任务运行的 CPU 位图; 第 N 位对应 CPU N。
79. pub cpus_allowed: usize,
80. /// 是否请求在安全调度点重新调度当前任务。
81. pub need_resched: bool,
82. /* 11. 杂项 */
83. pub clear_child_tid: usize, // 线程清理指针
84. pub vfork_completion: Option<Arc<VforkCompletion>>, // vfork 同步原语
85. pub personality: usize, // 进程个性化标志
86. pub locked_bytes: usize, // MAP_LOCKED 映射字节数
87. pub comm: [u8; 10],
88. }
```

除了以INITTASK为根的父亲关系进程树外，每个任务还会被全局管理器TID2TASK以B树的形式持有一份Arc引用，直到任务退出并被父进程wait回收。

3.1.2 系统调用 clone

对于clone，在复制TaskStruct后，根据clone传入的参数克隆任务结构体。对于线程，由于TaskStruct中的内存，文件描述符表等项目都是Arc引用，因此在clone时，可以自由选择字段是否与原线程指向同一个资源。这样可以淡化创建时进程和线程的专门区分，也可以避免额外PCB锁的开销。

我们的clone还支持写时复制，详见内存管理章节。

3.1.3 系统调用 exec

对于exec，其逻辑与rCore相似，更新memory_set，编辑auxv向量，构造上下文并返回用户态。

我们还实现了动态加载，详见内存管理章节。

3.1.4 内核工作线程

为POSIX软件计时器，网卡poll，磁盘定期回写等需要内核定期完成的任务，我们实现了内核工作线程支持。

其复用TaskStruct结构用于调度管理，初始化时在task_context中构造指定内核工作函数入口作为上下文：

```
1. impl TaskContext {
2.     // ...
3.
4.     pub fn goto_kernel_worker(entry: fn() -> !, kstack_ptr: usize) -> Self {
5.         //println!("goto_kernel_worker: kstack_ptr=0x{:x}", kstack_ptr);
6.         Self {
7.             ra: entry as *const () as usize,
8.             sp: kstack_ptr,
9.             s: [0; 12],
10.        }
11.    }
12.
13.    // ...
14. }
```

但不持有任何用户态资源，也不与任何其他进程产生父子关系。

工作线程需要有自己的工作循环，并在每次工作完成后主动调用睡眠/阻塞让出CPU，并且不得返回或尝试用trap_return()等进入用户态。除此以外，内核工作线程与用户线程在调度管理的视角看来是无异的。

3.2 调度策略

3.2.1 任务状态与切换

每个核有自己的调度循环，即run_tasks()。

由__switch完成内核态的task_ctx切换，即切换内核态的任务执行流。基于这个函数和包装好的schedule()可以完成从某个线程的内核态任务执行流切

回本核的idle执行流，即上次run_tasks()调用__switch的下一行代码，间接进行用户线程的状态切换和调度。

用户线程的状态转换图如下：

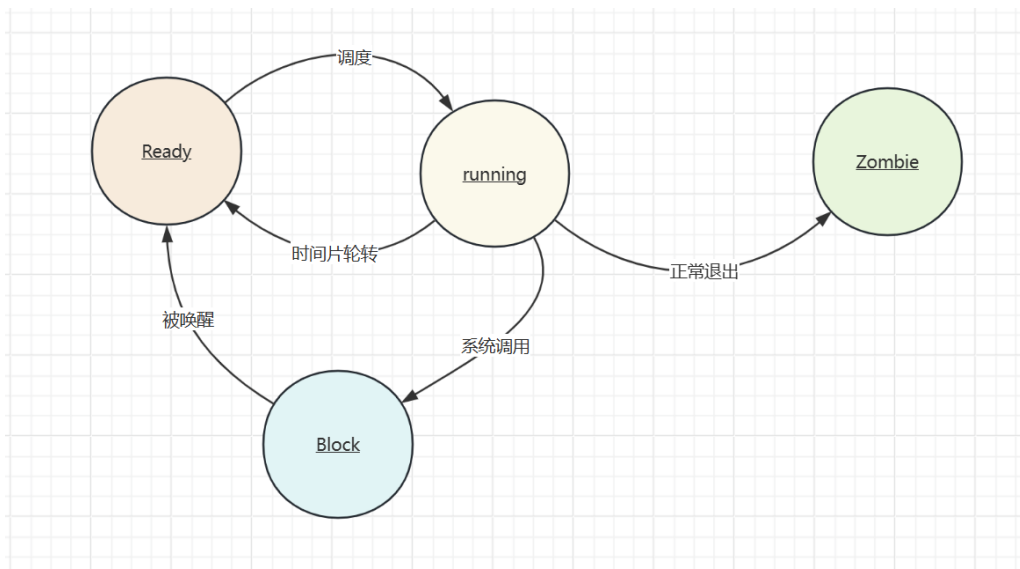


图 3.1 任务状态机

此外还会有BlockSaving这个过渡状态，主要是由于在多核调度环境下，futex_wait和futex_wake可能几乎同时发生，这导致futex_wake可能将刚进入阻塞但未保存号上下文的进程拿出队列唤醒，这可能导致恢复错误的上下文。

3.2.2 调度策略

我们沿用了rCore的run_tasks()函数作为调度主循环，但进行了大幅修改。

目前每个核都有自己本地的run_queue，即调度队列。任务在唤醒，创建，调度后会直接将任务会优先搜索是否有处于空闲状态的核，将任务放到空闲的核上，如若没有则会放回之前的核。

同时，任务也会在本队列为空时从其它核上偷取一个任务。（偷不到则会进入空闲状态）

核内调度的run_queue如下

```
1. pub struct Rqinner {
```

```

2.  /// 最高优先级的每 CPU stop 调度类状态。
3.  pub stop: StopRq,
4.  /// SCHED_DEADLINE 的每 CPU 运行队列。
5.  pub deadline: DeadlineRq,
6.  /// 普通、批处理及 SCHED_IDLE 用户任务使用的 CFS 运行队列。
7.  pub cfs: CfsRq,
8.  /// SCHED_FIFO 和 SCHED_RR 使用的实时运行队列。
9.  pub rt: RtRq,
10. /// 最低优先级的每 CPU idle 任务状态。
11. pub idle: IdleRq,
12. /// 除 per-CPU idle 任务外的可运行任务总数。
13. pub nr_running: usize,
14. }

```

run_queue中有自己的多级队列，每个队列中是基于时间片轮转运行的，优先级是自上向下选择。被阻塞唤醒为就绪态的任务会被放到deadline队列里，其余会被放到cfs队列中。

3.2.3 睡眠与阻塞/唤醒机制

在引入了wait、futex、epoll、ppoll等带有“等待”语义的调用后，我们发现早期作为过渡的睡眠轮询效率过低，应当实现可控高效的阻塞唤醒机制。为此，我们实现了通用的阻塞唤醒函数，可以将任务阻塞到特定队列或从特定队列中唤醒。阻塞队列的定义如下：

```

1.  pub type TaskControlBlock = TaskStruct;
2.
3.  pub struct WaitQueue {
4.      queue: VecDeque<Arc<TaskControlBlock>>,
5.  }

```

以此为基础，各阻塞源可以有不同的队列，还能根据队列确定“任务是否因该阻塞源阻塞”。当任务希望阻塞自身时会原子化地判断条件并插入队列；唤醒时则会原子地检查任务状态并唤醒。

下面讲解关键函数：

suspend_current_and_run_next(): 睡眠当前任务并触发一次调度。先将当前任务的状态改为Ready, 再schedule()切回idle, 然后在idle中将当前任务放入任务池并尝试获取下一个任务。这样做是为了保证的原子性, schedule()保存task_ctx前该任务将不会被其他核拿出执行。

block_current_and_run_next(): 阻塞当前任务并触发一次调度。先将当前任务的状态改为BlockSaving并放入阻塞队列, 再schedule()切回idle, 然后在idle中将当前任务的状态改为Blocked。这样做是为了保证阻塞的原子性, schedule()保存好task_ctx前, 如果有别的核希望通过wake_up_one()唤醒BlockSaving的任务, 将为其标记未决唤醒, 保存好上下文的时任务会因为该标志重新改为Ready并进入调度队列。

block_current_and_run_next_if(): 会额外传入一个闭包, 函数会在队列锁内原子地判断条件并决定阻塞/不阻塞。

wake_up_one(): 唤醒一个任务。判断任务状态是否为Blocked, 是则唤醒并从队列中取出; 如果是BlockSaving则标记未决唤醒; 其他状态则只出队不唤醒。本函数有几个衍生函数, 可以唤醒队列的全部任务等。

3.2.4 futex 结构

futex的阻塞唤醒可以复用上一节提到的机制, 关键在于队列的管理。我们为futex实现了一个全局队列管理器, 建立了键到队列的映射。

早期的实现中, 对于进程内私有语义, 为了避免进程冲突, 我们将futex队列的键定原子变量的物理地址。但由于写时复制机制, 可能出现新进程在cow前后对同一虚拟地址调用futex找错键的情况。因此我们采用了以虚拟地址加根页表基址为主的键。

而对于shared语义, 使用了三种键, 一种是mm, 采用根页表地址, 一种是AnonymousMapping, 采用maparea id, 一种是文件(内存映射), 采用inode no与偏移量。

3.3 睡眠相关

对于`nanosleep`，将任务放到全局睡眠队列，在每次`cpu`走到内核调度循环时更新一次时间戳，如果有任务时间到了则唤醒。将任务放到空核或原核的`deadline`队列中。

POSIX软件`Alarm`则由专门的内核工作线程定期执行唤醒。

3.4 信号处理相关

3.4.1 信号处理

任务在被唤醒、`trap`返回前会扫描任务本身的未屏蔽未决信号并进行处理；特别地，部分需要阻塞的系统调用因原子化语义需要，会在获取到锁后、阻塞前检查并处理致命信号。

ShellCore支持用户修改除`sigkill`和`sigstop`以外的信号屏蔽掩码，以及注册用户程序自定义的信号处理函数所在地址。（记录在`Task_Struct`的结构体中）

信号处理支持线程自己的信号处理以及进程内所有线程的信号处理。

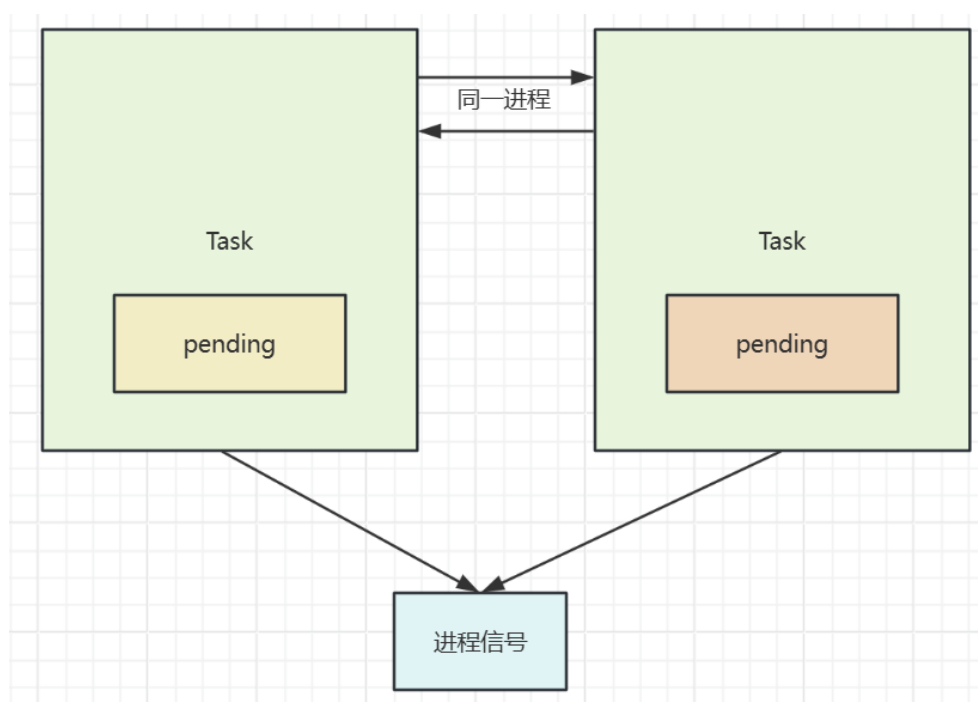


图 3.2 Task 信号数据结构示意图

我们为信号处理实现了上下文备份和恢复。`TaskStruct`中有专门的通过在跳板页中插入信号处理返回函数，并在进入用户信号处理函数前保存上下文，篡改`ra`寄存器，使用户在信号处理完成后返回时自动跳转到该函数，发起`rt_sigreturn()`系统调用请求，从而完成信号处理的上下文恢复。这里以`riscv`为例：

```
1. # 信号处理后恢复原本上下文
2. __call_sig_rt:
3.     li a7, 139
4.     ecall
```

在此实现下，也能够支持嵌套信号处理。

3.4.2 系统调用 `wait`

`wait`目前会将任务阻塞在自己`Taskstruct`中`siganl`的队列中，当子进程退出时会通过指向父进程的弱引用唤醒父进程，清空等待队列。父进程会再次观察是否成功等待到指定子进程，如果没有的话回到队列中再次阻塞。

RISC-V无论是内核还是用户均采用SV39模式建立页表映射，支持大页映射。

内核空间不再采用恒等映射，而是通过类似窗口映射的方式将物理内存映射到高半地址空间。在开启分页前的启动阶段，用临时的一级页表以1GB大页建立整个高半地址空间到0~256GB物理内存的映射，然后开启分页。而后则在新页表以2MB为单位细粒度地建立MMIO区域和物理内存的映射。2MB大页在内存访问速度和管理粒度间取得了平衡。

用户态的内存模型与初赛基本一致，在内核的页帧管理器中分配4KB页面（暂时不考虑memfd等大页情况），通过页表建立映射。与初赛不同的是，每个用户根页表的[256, 512)均直接复制自己已经完成初始化的内核根页表，在trap期间不再切换根页表基址到内核空间，仅在任务切换且内存空间发生实质改变时修改satp。这样做的好处是：qemu tcg会在每次写入satp时执行全量TLB刷新，不切换页表可以尽可能地减少TLB miss。

4.1.2 LoongArch

LoongArch的地址空间关系如下图：

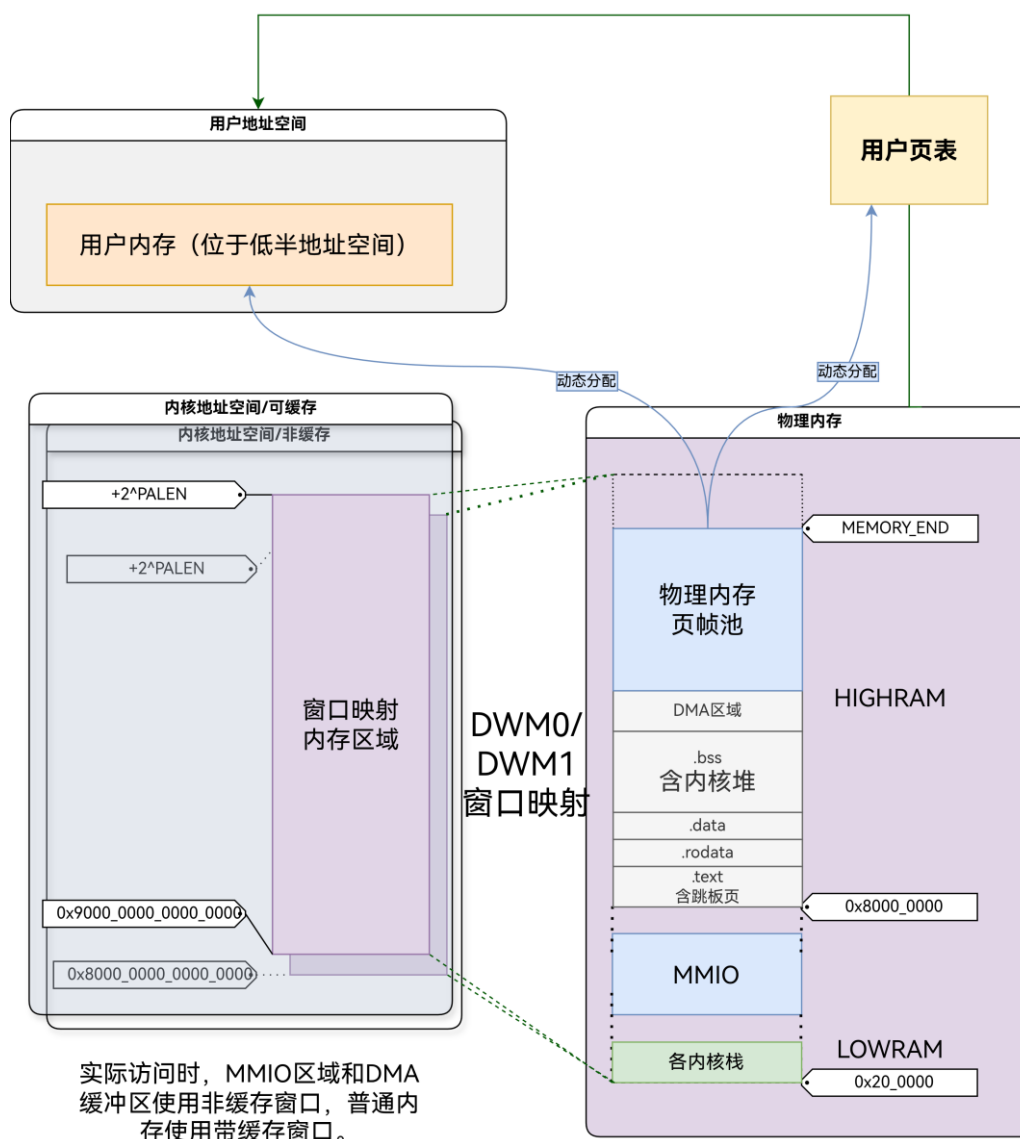


图 4.2 loongarch 地址空间映射关系图

通过配置DWM的权限位，LoongArch下陷入内核态后会自动全程采用映射窗口访存；用户态则同样用页表，与riscv基本一致，不过需要额外实现tlb重填和异常处理标脏等。

为了上层实现的方便，配置LoongArch页表也为9+9+9+12的三级页表/4K基本页，并以此为基础实现了RISC-V具有的页表相关方法，例如PTE flags权限抽象，便于软件页表访问/缺页处理等的实现能够与具体架构去耦合。

由此，对于用户内存管理，两种架构大体上是一致的。

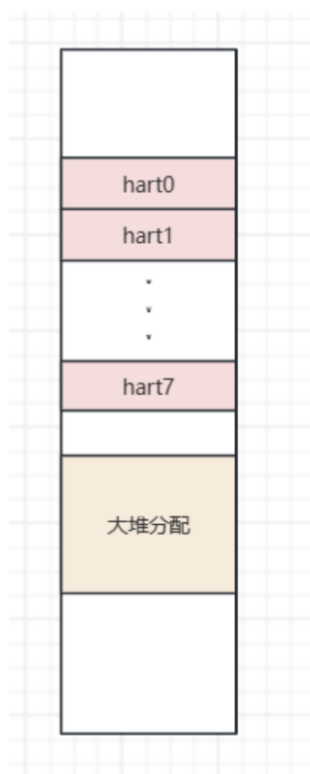
4.2 内存分配

4.2.1 内核堆分配

rCore的全局单堆不支持并行分配，并发等待开销大。于是我们的内核堆分配采用基于伙伴系统的大小堆实现。

堆区位于内核.bss段，我们为每个核心开辟了一个自己的小堆区，并开辟了一个共用大堆，每个小堆和大堆均由一个buddy_system_allocator库的伙伴系统分配器管理。

每个核心在需要堆分配时优先使用自己的小堆，仅当小堆满时回退到使用大堆。



4.2.2 页帧分配与管理

在我们的内核中，不论是用户内存还是页缓存，均以页帧Frame为单位，FrameTraker是页帧管理的基本数据结构。与rCore不同的是，我们通过Arc为其实现独立的低开销引用计数，为COW/pin页等的实现提供了基础：

```

1.  /// 用 Arc 替代了原有的引用计数器
2.  pub type FrameTracker = Arc<Frame>;
3.
4.  /// tracker for physical page frame allocation and deallocation
5.  /// 引用计数挪至外层 Arc 包装
6.  pub struct Frame {
7.      /// physical page number
8.      pub ppn: PhysPageNum,
9.      /// 页大小
10.     pub page_size: PageSize,
11. }
```

我们为Frame实现了Drop Trait：在FrameTraker引用计数归0被释放时，将会自动访问页帧管理器回收该页。

为了支持大页/可变页大小分配，并且不丢失分配的灵活性，我们借鉴伙伴系统的思想，改进了rCore的StackFrameAllocator。用4KB/2MB/1GB三种页分别收集释放回收页，并在分配时优先使用回收页。本回收栈为空时，尝试找更大的页并拆分，失败则把current向上对齐并分配，对齐导致的碎片则收集到回收栈。算法流程如下：

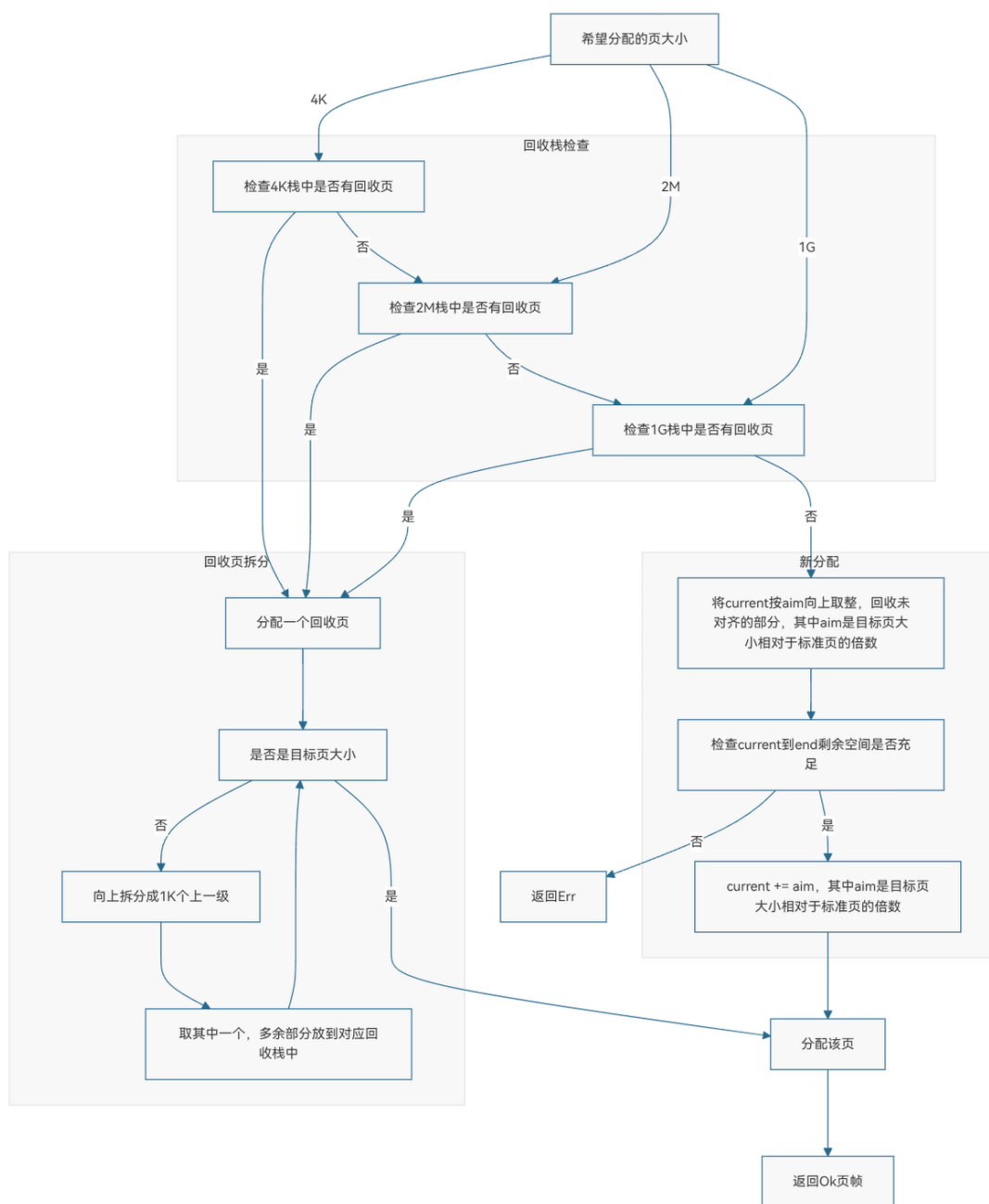


图 4.3 页帧分配算法流程图

4.3 高级特性与缺页处理

COW: clone在复制内存时，对于私有可写映射，不复制数据，只复制FrameTracker，并且将父子进程的相应页表均改为不可写。当用户发生写入时会触发PageFault，由异常处理完成数据复制。

映射懒分配：当调用mmap获取匿名页或者文件页时，我们不会立即分配页面并映射，而是标记mmap的共享/文件/权限信息，相应页表项留空。这样，当用户访问相应地址时，会触发缺页，由缺页处理程序分配页面并补全映射，文件映射则分配缓存页并只读映射。

exec懒加载：内核只读取elf头，按程序段表插入虚拟内存段并设置backing file但不建立映射；当程序发生缺页时，以共享文件映射的形式从全局页缓存管理器获得相应页帧的引用，并在页表中建立只读映射；当页面发生写权限异常时，再新分配一页复制文件缓存数据，重新在页表中建立可写映射。

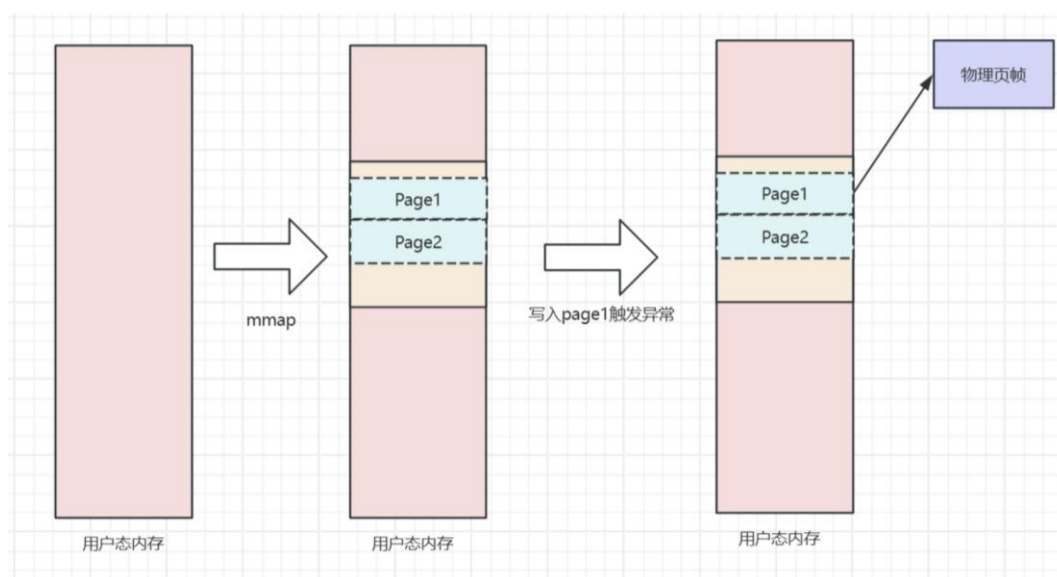


图 4.4 缺页处理示意图

4.4 页缓存管理

我们在早期简单块缓存和中期页/块二级缓存基础上做了大幅度改进，使其既能作为块设备缓存加速磁盘访问，又能作为页缓存被用于文件读取或零拷贝文件映射。页缓存的管理单元为PageCache：

1. /// 页缓存

```

2. ///
3. /// 合并了原本的 SharedPageCache 和 BlockCache 的功能
4. /// 当前实现规定为 4KB
5. pub struct PageCache {
6.     inner: Mutex<PageCacheInner>,
7.     block_id: usize,
8.     block_device: Option<Arc<dyn BlockDevice>>,
9.     /// 数据块缓存还是元数据块缓存（淘汰时按类分别限制）
10.    is_data: bool,
11.    /// 最近一次访问的全局时间戳（仅用于淘汰决策，无需精确）
12.    last_used: AtomicU64,
13. }
14.
15. pub struct PageCacheInner {
16.     pub frame: FrameTracker,
17.     pub dirty: bool,
18.     pub state: CacheState,
19.     /// 缓存已作废：对应物理块已被释放，禁止再写回
20.     pub discarded: bool,
21. }

```

其核心字段为 Inner 中的 FrameTracker，其余字段和锁多用于同步/互斥访问和回写。正是由于前文提到的 FrameTracker 引用计数这一特点，页缓存管理中，既能通过持有 FrameTracker 读写（pin 页）来保证内存安全，又能通过简单的去包装，将文件缓存页直接映射到用户空间。

所有的页缓存都由全局唯一的页缓存管理器进行管理。其结构体定义如下：

```

1. /// 页缓存管理器
2. ///
3. /// 管理所有文件系统数据块和元数据块的缓存。
4. /// 从原有 SharedPageCacheManager 修改而来，
5. /// 并且合并了原本 BlockCacheManager 的功能
6. ///
7. pub struct PageCacheManager {
8.     /// (ino, logical_block_id) -> physical_block_id
9.     page_cache_id_map: Mutex<CacheIdMap>,
10.    /// physical_block_id -> cache
11.    page_cache_map: RwLock<BTreeMap<u64, Arc<PageCache>>>,
12.    /// 元数据缓存条目数（避免每次 miss 都全表扫描判断是否超限）

```

```
13. meta_count: AtomicUsize,  
14. /// 数据缓存条目数  
15. data_count: AtomicUsize,  
16. /// 元数据和数据缓存的轮转扫描起点，避免候选长期偏向低物理块号  
17. meta_scan_cursor: AtomicU64,  
18. data_scan_cursor: AtomicU64,  
19. }
```

`page_cache_id_map`记录了从(inode号, 磁盘逻辑块号)到物理块号的双向映射。正向映射用于文件访问时快速查询指定逻辑页的块号, 反向映射则用于页缓存被换出时的快速反向查找(用于移除正向映射)。`page_cache_map`则直接记录了当前缓存的所有缓存页, 以物理块号为键。

由于我们的内核目前采用固定单盘单份区挂载, 并且保证了所有文件系统不同文件的inode号互斥, 上述以inode号作为区分的实现时正确且简洁的。

页缓存每次被写入时都会被标记为脏。当主动调用`sync`/触发定期全量写回/内核关机时, 所有脏缓存页都会被写回磁盘。定期回写由专门的工作线程完成。

当内存不足或定期检查时发现缓存数量超过上限时, 会触发页缓存换出, 相应项的索引会被取消注册, 页帧将在写回后被回收。对于定期淘汰, 我们采用了近似LRU算法: 在页缓存访问时调用一次`touch()`, 更新总访问次数和本页访问序号, 序号越大代表越新; 每次淘汰时, 只扫描当前游标后一定数目内数个连续窗口, 取每个窗口序号最小的缓存页尝试淘汰; 完成后更新一次游标, 游标到达末尾时又回到开头, 不断循环。而对于内存压力淘汰, 则直接线性全表扫描, 直到回收了足够多的页。

4.5 Pin 页机制

有时, 我们希望“固定(Pin)住”某一个页缓存以安全地通过地址零拷贝访问文件内容, 或是希望在访问某一页内存时保证其暂时不被释放。具有引用计数和 `drop` 自动回收的 `FrameTracker` 天然能支持这个语义。

我们的内核在许多地方使用了这个机制来提高效率和保证并发安全，下面将举一些具体例子。

`Ext4Inode` 会持有相应页缓存对象的引用，在需要读写 `inode` 元数据时直接访问相应缓存页地址；

内核有时会希望访问用户传入的指针，在进行软件页表翻译期间，先尝试获取相应页面的 `FrameTracker`，再进行页表翻译和数据访问，这样做既能避免持锁读写数据，又能保证期间内即使页面被其他并行任务取消映射，也不会访问到已经被释放的内存；

`unmap` 取消映射后，在 `area` 写锁外刷新 `tlb`，`tlb` 刷新完成前始终持有相应页帧，避免其被释放后其他核因陈旧 `tlb` 项访问到这段内存。

第 5 章 文件系统

5.1 文件系统概述

文件系统的架构层与 `rCore` 很相似，大体如下：

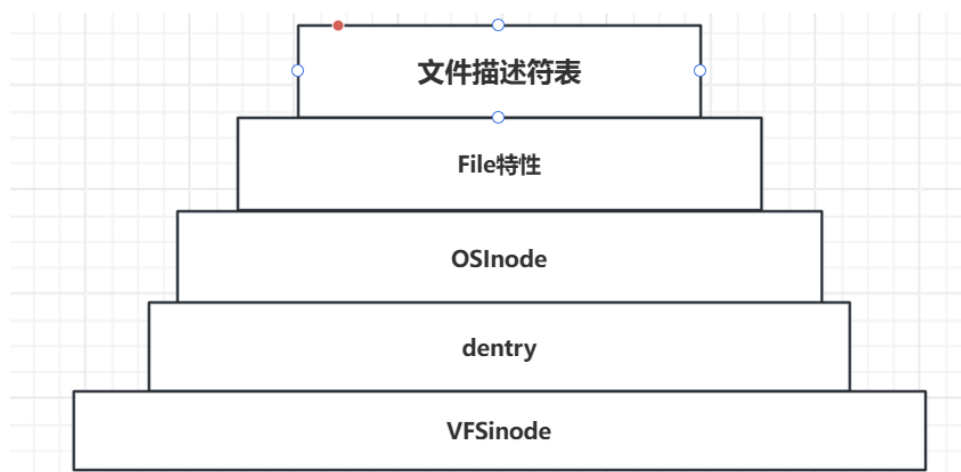


图 5.0 文件系统架构层

由于rCore初始不支持多级目录结构，我们添加了一层dentry层用于充当多级目录结构树中的索引节点，同时dentry层有对底层vfsinode的引用，用于与底层文件系统交互。其定义如下：

```
1. pub struct Dentry {
2.     pub inode: Arc<dyn VfsInode>,
3.     location: Mutex<DentryLocation>,
4.     /// POSIX 记录锁表；同一dentry（同一文件）的所有打开者共享
5.     pub file_locks: Mutex<alloc::vec::Vec<super::FileLock>>,
6.     /// 对当前目录的操作（如插入、删除、查找）都应加此锁
7.     /// 保证不发生数据竞争
8.     pub namespace_lock: Mutex<()>,
9.     /// 子目录（文件）正缓存
10.    ///
11.    /// 目标文件系统的子目录和文件的缓存，
12.    /// 它们是从 backing filesystem 查找的结果。
13.    pub children: Mutex<BTreeMap<String, Arc<Dentry>>>>,
14.    /// 挂载子目录
15.    ///
16.    /// 被挂载到当前目录的子目录和文件的缓存，
17.    /// 它们是 overlay filesystem 的结果。
18.    pub mounted_children: Mutex<BTreeMap<String, Arc<Dentry>>>>,
19.    /// 子目录负缓存
20.    ///
21.    /// 这些名字在 backing filesystem 中不存在，查找失败时插入到这。
22.    /// 新增的子目录或文件会清除对应的负缓存，并添加到正缓存中。
23.    negative_children: Mutex<BTreeMap<String, ()>>>,
24.    /// 记录当前 inode 已经不再被任何父目录引用
25.    ///
26.    /// 当 inode 被 unlink 后，dentry 仍然可能被其他地方引用（如打开的文件描述符），
27.    /// 但它已经不再属于任何目录树的一部分了。
28.    unlinked: AtomicBool,
29.}
```

在内核初始化文件系统时，会自动读取主硬盘分区的根目录Inode并将其实例化为ROOT_DENTRY，所有的路径访问都将在这棵以根目录为根的目录树上进行。

5.2 文件描述符表

ShellCore的文件描述符表依旧采用了由TaskStruct记录的形式：

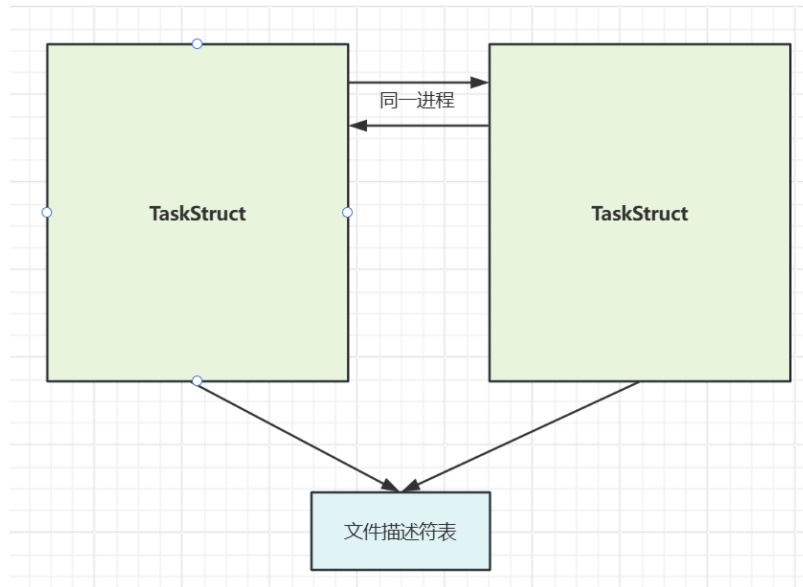


图 5.1 文件描述符表实现

文件描述符表是 `Vec<dyn File>`，这与 `rCore` 设计一致。关于 `File`，下一节会详细描述。

为适配 `Linux ABI`，关于任务中对于文件描述符向量的最大长度以及文件大小等限制的字段存储在 `TaskStruct` 中，具体可见 `TaskStruct` 结构体定义。

5.3 File Trait

参考“一切皆文件”的 `Linux` 哲学思想，我们定义了 `File` 特性，其固有方法为 `read/write`，包括 `stdio`，管道，`Socket`，`OSInode` 等。

在我们的内核中，这里的 `File` 即为可以被用户打开到文件描述符表，并通过系统调用操作的“文件”。

5.4 vfs 层与 ext4 文件系统

5.4.1 vfs 抽象

VfsInode是一个特征，任何VfsInode泛型都可以被包装为Dentry而挂载到目录树上；同时Dentry既作为目录对象可以被遍历，又可以被进一步包装为OSInode，作为File被用户打开和使用。

5.4.2 ext4

ext4文件系统是相较于rCore以外额外添加的，但对于底层文件系统的抽象层也参考了rCore的文件系统架构实现。即自上向下为 Ext4Inode -> Ext4FS -> 页缓存 -> 块设备。

由于目前的多磁盘需求不大以及时间限制，部分内容暂时实现为使用全局唯一 BLOCK_DEVICE，但由于我们的实现中每个 Ext4FS 的块设备是独立字段，扩展为动态挂载是比较容易的，只需修改少量文件系统/块设备初始化相关代码。

另外，我们初步支持了将 loop 设备挂载为 ext4 磁盘，但暂未经全面验证。

我们通过Ext4FS来抽象一个ext4分区：

```

1. #[allow(dead_code)]
2. pub struct Ext4FS{
3.     pub block_dev: Arc<dyn BlockDevice>,
4.     pub superblock: Ext4SuperBlock,
5.     pub block_groups: Vec<Arc<Mutex<Ext4Group>>>,
6.     /// Serialize allocation and raw metadata updates. ext4 metadata is a
7.     /// cross-block transaction even when this implementation has no journal.
8.     pub metadata_lock: Mutex<()>,
9.     /// inode 缓存表: ino -> Weak<Ext4Inode>
10.    ///
11.    /// 从磁盘读取 inode 前先查表;
12.    /// 从磁盘中读取一个 inode 时, 会将其注册(缓存)到表中;
13.    /// 后续访问时, 若缓存中存在则直接使用。
14.    /// 所有已注册表项不应当被主动删除。
15.    pub inodes: Mutex<BTreeMap<u32, Weak<Ext4Inode>>>,
16. }
```

其中记录了内核通过Ext4FS::open(dyn BlockDevice)来打开一个ext4分区。打开时，先从磁盘中读取超级块，然后构造块组vector。接着可通过Ext4FS::get_inode()在刚刚打开的ext4fs上打开一个指定inode号的磁盘文件为Arc<Ext4Inode>对象（特别地，所有Inode对象在构造时都会被注册到Ext4FS中的Inode对象弱引用缓存表，并优先查表而非总是从磁盘读取，防止并发重复读取造成数据不一致）。

Ext4Inode的定义如下：

```

1. pub struct Ext4Inode {
2.     /// Inode 编号
3.     pub inode_id: u32,
4.     /// 文件大小快照；作者在更新磁盘inode 后发布，读者无需获取inode 页锁。
5.     size: AtomicU64,
6.     /// 写入锁：原子化文件内容的写入操作，避免多个线程同时修改文件数据
7.     pub write_lock: Mutex<>,
8.     /// 块映射锁：原子化逻辑块到物理块映射的查找、分配和删除操作
9.     pub block_map_lock: Mutex<>,
10.    /// Serializes a whole directory read-modify-write operation. The file
11.    /// write lock alone only protects the final write, leaving two directory
12.    /// scanners able to commit stale snapshots over each other.
13.    dir_mutation_lock: Mutex<>,
14.    /// 被固定的inode-table 页及inode 在页内的字节偏移。
15.    disk_inode: PinnedDiskInode,
16.    /// inode 代数：记录创建时的磁盘代数，防止ino 被释放并复用后，
17.    /// 失效的旧对象误释放新文件
18.    generation: u32,
19.    /// 块设备
20.    pub fs: Arc<Ext4FS>,
21.    /// 父目录Inode 编号（可选）
22.    pub parent: Option<u32>,
23.    /// A bounded, byte-exact cache for names recently enumerated or looked up
24.    /// in this directory. ext4 filenames are byte strings, not Rust strings.
25.    dir_lookup_cache: Mutex<DirLookupCache>,
26. }
```

打开时，会先从磁盘上读取并缓存相应inode元数据所在页，然后将页缓存对象的一份Arc引用固定到本inode对象中，后续获取/修改元数据时可以在页缓存锁内将其重新解释为&Ext4InodeDisk，实现零拷贝访问。而Ext4Inode实现了VfsInode trait，由此能够被包装为实现了File trait的OSInode，最终作为

文件被内核操作或被用户打开。而上层抽象的文件读写等请求，经过系统调用，又经OSInode封装的转发，最终会落到Ext4Inode的具体方法实现。

Ext4Inode的主要方法有：

1. `raw_read_at()`从文件指定偏移读在磁盘（块设备）上的数据；
2. `raw_write_at()`同理，写；
3. `add_dir_entry()`（针对目录文件）向文件中插入一个目录项；
4. `delete_dir_entry()`同理，删除；
5. `truncate()`改变大小；
6. `find_physical_block()`从数据逻辑页号找到其在磁盘（块设备）上的物理页号；
7. `add_extent_entry()`添加extent项，必要时分裂树；
8. 一些判断文件类型的方法；
9. 一些校验相关方法。

`raw_read_at()`先从磁盘读取inode信息（如果是符号链接直接将其拷贝到缓冲区并返回，否则继续），然后循环通过`find_physical_block()`尝试找到指定偏移数据的物理块号，读取数据，直到填满缓冲区。

`raw_write_at()`同理，不过在`find_physical_block()`失败时分配新块。根据模式行为有所不同：传统模式，插入直接块（目前的简化实现暂不支持传统模式下的间接块）；extent模式，调用`add_extent_entry()`将刚分配的数据块插入extent树。

`find_physical_block()`根据逻辑块号和文件模式（传统/extent）遍历树找到物理块号。

`add_extent_entry()`是其中最复杂的方法，本质上是一个B+树的插入与分裂算法。在插入时：

1. 先自上（`i_block`，根节点）而下走到叶子层，找到插入位置（如果有逻辑页号和物理页号都连续的已有节点，与其合并）；

2. 如果下层需要分裂，则分配新块，将下层需要分裂的节点分裂到其中，本层则插入一个项指向新块；
3. 如果本层也已满，则把分裂信息向上传递，让上层节点处理。特别地，根节点单独重写逻辑。

由于磁盘操作相较于内存操作步骤更为繁琐，实现起来较为困难。权衡了实现的可读性与开销后，我们最终决定采用堆分配缓冲区的递归调用。这样做的基础是严格按照 `ext4` 规范，深度不超过 5 次，并且调用栈上只保存几个整数变量/堆指针，不分配数据缓冲。

`add_dir_entry()` 遍历所有已有条目，尝试在插入新目录项到冗余空间中（如果足够大），如果没找到则分配新块扩展文件长度，再插入新条目。

`delete_dir_entry()` 遍历所有目录项，找到第一个（实际上也是唯一一个）与目标同名的目录项，按文件类型执行删除。

简明起见，上述几个方法描述中的校验步骤均被省略，实际代码中有较为完整的校验，详见 `os/src/ext4fs/ext4inode.rs`。

其对 `VfsInode` trait 的实现还有一些额外方法，本质上大都是对上述方法的抽象封装。

5.5 文件页缓存使用

文件缓存主要针对底层为磁盘硬件的 `ext4fs`。对于 `tempfs`，涉及页缓存的部分则被实现为直接操作内存文件数据页。

不论是 `File` 还是 `VfsInode`，其 trait 中都有 `get_shared_page()` / `get_file_page()` 方法。对于实现了 `File` 的 `OSInode`，其将会被转发到其 `VfsInode` 的同名方法。而如果 `VfsInode` 实例是 `Ext4Inode`，这两种方法均会被实现为：先尝试从页缓存管理器查询并获取缓存，如果缓存命中则直接返回缓存页结构；如果不命中则查询对应物理块号，注册相应 `ino` 和逻辑块号（文件内部偏移）到物理块号的映射，并为该物理块号分配内存中的相应页帧。二者的区别在于：`get_shared_page()` 会在相应逻辑位置的物理块不存在时，从文件系统中分配新的磁盘块并注册到管理器；而 `get_file_page()` 会直接返回 `None`。

File和VfsInode的普通读写均会先查询页缓存，缓存命中直接拷贝到缓冲区，而当页缓存不命中时调用“原始”读写，由底层文件系统先将数据读到缓存页（同时会完成注册），再拷贝到缓冲区。

特别地，对于数据直接位于i_block的快速符号链接，不分配页缓存，直接操作元数据缓存。

为了保证和truncate不发生并发冲突，以inode为单位引入了map_lock读写锁，保护对逻辑块号 -> 物理块号映射的访问。

第 6 章 系统 IO 与板卡支持

6.1 IO 抽象

我们为块设备定义了BlockDvice特征，其包含(raw_)read/write_block()共计四种方法，分别为经过/不经过缓存的读/写物理块。为特定物理磁盘或loop设备等实现BlockDvice Trait，即可让上层文件系统等访问其中的数据。不过目前我们还采用全局单盘实现，磁盘动态挂载等的支持有待完善。

网络设备则同样定义了EthernetDevice特征，提供帧的收发能力；网络层通过 SmoltcpDevice 对其进行包装，并实现 smoltcp 的 phy::Device、RxToken 与 TxToken Trait，从而将底层驱动提供的 receive_frame() / transmit_frame() 接口适配为 smoltcp 可直接使用的设备接口，实现协议栈与具体网卡驱动。

6.2 VisionFive2 支持

6.2.1 SD 卡支持

由于SD卡读取写入采用关DMA的方式，PIO 方式，因此数据搬运主要由CPU 与 SDIO 控制器配合完成。

CPU通过SDIO1 控制器完成命令通信和数据的收发暂存，通过写入CMDARG和CMD寄存器来完成命令通信，通过SDIO1 控制器的 RESP0 ~ RESP3 寄存器接收返回命令响应，通过 DATA寄存器读写SDIO1 内部 FIFO 中的数据。

数据读写过程中，CPU 通过轮询 SDIO1 的状态寄存器判断 FIFO 中的数据量、传输完成状态及错误状态。向上包装成了SdBlockDevice，把 sector 组合成block，增加锁后供上层读取。

同时做了分区处理，该系统只需要使用 SD 卡上的 ext4 分区，只做了轻量化的处理，为读取 SD 卡 sector 0 的 MBR 分区表，获取各分区的起始 LBA，并检查 ext4 superbblock的魔数确定 ext4 分区。让上层看到的 block 0 从 ext4 分区起点开始

6.2.2 网卡支持

DWMAC 内部开启了 DMA，CPU 不再直接负责网络数据在网卡与内存之间的搬运，而是维护 RX Ring 和 TX Ring，通过 DMA Descriptor 描述数据缓冲区的地址、长度和状态，并通过 OWN 位在 CPU 与 DMA 之间转移 Descriptor 的所有权，由 DMA 完成网卡与内存之间的数据交换。

CPU 通过写 DWMAC 内部的 MDIO 控制寄存器对 PHY 进行管理，MDIO 控制逻辑根据其中的 PHY 地址、寄存器地址和读写操作，自动通过 MDIO 总线访问 PHY。

MAC 与 PHY 之间通过 RGMII 进行网络数据传输，并通过 MDIO 对 PHY 进行配置和状态读取。

上板初始化时还需要打开 GMAC 各部分所需的 AHB、AXI、TX 等时钟，并解除相关模块的复位状态。其中 AHB 主要用于寄存器访问，AXI 用于 DMA 访问内存。

6.3 龙芯 2k1000 支持

6.3.1 板卡内存特性适配

2k1000物理板卡整体特性与qemu类似，内存布局等略有不同，按照uboot报告的内存定义完成配置即可。出于简单起见，我们没有用设备树，而是暂时采用条件编译+独立常量定义的方式，为每个设备配置内存。

不过，其与qemu最大的差异在于：不支持非对齐访存！我们发现默认参数下编译的代码，在qemu上运行正常，而在2k1000开发版上会崩溃；进一步调试发现，`println!`宏输出`{:#x}`时，`fmt!`会让内核立即崩溃，并且无任何错误信息。我们关闭了trap初始化，发现uboot的异常处理报ALE（非对齐访存异常），阅读《龙芯架构参考手册》的相关章节：我们首先决定设置MSIC寄存器主动打开非对齐访存，然而没有解决。查资料发现，2024年已有队伍报告了这个特性<https://gitlab.eduxiji.net/educg-group-26011-2376549/T202410699992496-312/>。参考了他们的解决思路，我们为内核编译打开了`build-std`和`-ual`，同时为用户程序提供了ALE异常的逐字节软件处理，最终解决了问题。

此外，为硬件板卡专门实现了内存屏障函数dma_barriar()（实际上就是dbar 0）来确保MMIO/DMA访问时的顺序一致性。

6.3.2 SATA 磁盘

2K1000板卡采用PCIE总线识别，AHCI控制的SATA SSD。我们的早期实现选择了保留由uboot初始化的AHCI控制器BAR并直接使用。

初始化磁盘按照扫描->复位->分配DMA区域->配置->启动的整体流程完成。先在PCI总线上扫描，找到AHCI控制器的BAR地址并将其实例化为AHCI_CONTROLLER，然后通过AHCI控制器扫描找到第一块SATA磁盘端口，并确认其签名符合ATA磁盘且已经建立通信。随后关闭端口引擎，关中断，清除旧的错误位和中断位等，接管硬件。然后按照下面的布局，从DMA内存管理器分配一块连续的DMA内存区域：

```
1. // dma 区域布局偏移设置
2. const PORT_DMA_SIZE: usize = 0x2000;
3. const COMMAND_LIST_OFFSET: usize = 0x0000;
4. const RECEIVED_FIS_OFFSET: usize = 0x0400;
5. const COMMAND_TABLE_OFFSET: usize = 0x0500;
6. const DATA_BUFFER_OFFSET: usize = 0x1000;
7. const DATA_BUFFER_SIZE: usize = PORT_DMA_SIZE - DATA_BUFFER_OFFSET;
8. const COMMAND_SLOT: usize = 0;
9. const COMMAND_SLOT_MASK: u32 = 1 << COMMAND_SLOT;
10. const ATA_COMMAND_TIMEOUT_MS: usize = 5000;
11. const DMA_COMPLETION_GRACE_MS: usize = 10;
```

```
1. /// 单个端口所使用的DMA 区域布局
2. ///
3. /// 所有字段均为提供给HBA 的物理地址，CPU 访问时需要转换到非缓存DMW
4. #[derive(Clone, Copy)]
5. struct AHCIPortDmaLayout {
6.     /// Command List 物理地址，写入 PxCLB/PxCLBU，必须按 1 KB 对齐
7.     ///
8.     /// 命令列表，有 32 个 slot（每个 32B），一个 slot 中装一个命令头，共 1KB
9.     pub command_list: PhysAddr,
10.    /// Received FIS Buffer 物理地址，写入 PxFB/PxFBU，必须按 256 B 对齐
11.    ///
12.    /// HBA 会将硬盘返回的 FIS 写入此缓冲区
13.    pub received_fis: PhysAddr,
```

```

14.  /// slot 0 Command Table 物理地址, 写入 Command Header 的 CTBA/CTBAU
15.  ///
16.  /// - CFIS(0x00,64B)
17.  /// - ATAPI(0x40,16B,ATA 填 0)
18.  /// - PRDT(0x80, 每项 PRD 16B)
19.  /// 目前只初始化 slot 0
20.  pub command_table: PhysAddr,
21.  /// 命令数据缓冲区物理地址, 后续由 PRDT Entry 引用
22.  ///
23.  /// 放读写数据的缓冲区, 目前暂时固定 4KB, 后续改为动态分配并支持大块预
    读
24.  pub data_buffer: PhysAddr,
25. }
```

分配完成后, 往命令列表的第 0 槽写进一个空命令头, 然后把命令列表和接收 FIS 缓冲区的物理地址分别写入 PxCLB/PxCLBU 和 PxFB/PxFBU 四组寄存器, 然后先后启动 FIS 接收引擎和命令列表引擎。最后向磁盘发送 IDENTIFY DEVICE (0xEC) 命令, 获取磁盘信息并保存到结构体字段中。

当需要读磁盘时, 则校验请求并构造一份读命令放先前分配的 command_table 区域。目前的实现中, PRDT 长度被固定为 1, 指向预先分配的 data_buffer 区域 (还未做成 0 拷贝地直接读写给定缓冲区/多块缓冲区)。构造好后通过 AHCI 向磁盘发送读请求, 等待完成并确认无错误后读 buffer。值得注意的是, 我们发现 PRDBC 有时不会被硬件更新, 所以即使检测到 PRDBC 为 0, 只要磁盘返回无错误同样返回成功。

写磁盘与读磁盘类似, 在 buffer 中准备好数据后构造一份写命令发给磁盘, 等待完成后返回写入成功。目前的早期实现不太稳定, 为了调试我们加入了读回校验, 确保写入成功。

6.3.3 GMAC 以太网控制器

2K1000 的 GMAC 以太网控制器不太好找到公开的详细资料, 我们的常量定义等部分参考了 U-Boot 和 2025RocketOS 的实现。下面是 DMA 布局的定义:

```

1.  /// 2K1000 GMAC DMA 相关常量
2.  ///
3.  /// DMA 寄存器 MMIO 区域起始偏移
4.  const DMA_REG_OFFSET: usize = 0x1000;
```

```
5. /// DMA 描述符环大小, 每个环有 16 个描述符
6. const RING_SIZE: usize = 16;
7. /// 每个描述符有 2KB 缓冲区
8. const PACKET_BUFFER_SIZE: usize = 2048;
9. /// 定义 DMA 缓冲区布局
10. ///
11. /// 为描述符预留一个标准页
12. /// 其中包含 TX 描述符表 + RX 描述符表
13. const DESCRIPTOR_PAGE_SIZE: usize = PAGE_SIZE;
14. /// 一个环的所有缓冲区总大小
15. const RING_BUFFER_SIZE: usize = RING_SIZE * PACKET_BUFFER_SIZE;
16. /// TX 缓冲区起始偏移
17. const TX_BUFFER_OFFSET: usize = DESCRIPTOR_PAGE_SIZE;
18. /// RX 缓冲区起始偏移
19. const RX_BUFFER_OFFSET: usize = TX_BUFFER_OFFSET + RING_BUFFER_SIZE;
20. /// DMA 内存区域总大小
21. const DMA_LAYOUT_SIZE: usize = RX_BUFFER_OFFSET + RING_BUFFER_SIZE;
22. /// 按页计
23. const DMA_LAYOUT_PAGES: usize = (DMA_LAYOUT_SIZE + PAGE_SIZE - 1) / PAGE_SIZE;
```

初始化时, 先在PCI总线上扫描找到GMAC控制器BAR, 然后分配DMA内存, 复位并设置轮询, 接着读取/设置MAC地址, 再配置描述符环设置和DMA区域, 配置速率/双工模式后启动MAC和DMA。整体思路依然是扫描->复位->分配DMA区域->配置->启动。

收包时, 由硬件自动把收到的包放到RX缓冲区, 软件轮询从当前idx开始, 检查OWN位为1并读出数据, 然后重新置OWN为0并更新idx。

发包时, 将数据帧构造填入TX缓冲区, 通知硬件开始DMA, 然后轮询等待至OWN位变为0。

以此为基础实现网卡特征, 硬件网卡便能够被上层软件使用。

第 7 章 网络相关

7.1 底层网卡驱动与 smoltcp 适配

网络设备针对软硬件不同架构采用分层适配方式。如在IO抽象中所说，底层网卡驱动首先实现内核统一的 `EthernetDevice` 接口，向上提供以太网帧级的收发能力；随后由 `SmoltcpDevice<D>` 对底层驱动进行包装，并实现 `smoltcp` 提供的 `phy::Device Trait`。

`SmoltcpDevice` 通过 `receive()`、`transmit()`和 `capabilities()` 向 `smoltcp` 描述设备能力，其中接收和发送分别通过 `RxToken` 与 `TxToken` 完成。`TxToken` 最终调用底层设备的 `transmit_frame()`，而 `receive()` 则通过 `receive_frame()` 获取底层收到的以太网帧。

7.2 网络协议栈与接口管理

系统当前维护两个网络接口：

LO_IFACE：基于 `smoltcp` 提供的内存 `Loopback Device`，配置 `127.0.0.1/8`，用于本地回环通信。

NET_IFACE：绑定实际底层网卡设备，用于与外部网络通信。

网络协议栈的推进由 `net_poll()` 完成。该函数分别调用 `loopback` 和物理网络接口的 `Interface::poll()`，由 `smoltcp` 完成协议解析、TCP 状态机推进、报文组装以及底层数据收发

```
1. let lo_active =
2.   lo_iface.poll(timestamp, &mut *lo_device, &mut sockets);
3. let eth_active =
4.   eth_iface.poll(timestamp, &mut eth_device, &mut sockets);
```

7.3 Socket 抽象

TCP、UDP、RAW 等 `smoltcp socket` 统一存放于全局 `SocketSet` 中，并通过 `SocketHandle` 进行索引，内核在 TCP/UDP socket 外继续封装文件接口，使其可以通过 `fd` 和 `read/write`、`sendto/recvfrom` 等系统调用访问。TCP socket与UDP socket封装类似，这里以TCP为例

TCP socket 创建时会分配 `smoltcp` 收发缓冲区，并将其注册到全局 `SocketSet`。读取时首先判断 `smoltcp` socket 是否存在可接收数据，随后通过 `recv_slice()` 将协议栈接收缓冲区中的数据复制至用户缓冲区：

```
1. if socket.can_recv() {  
2.     let mut temp_buf = vec![0u8; buf.len()];  
3.     match socket.recv_slice(&mut temp_buf) {  
4.         Ok(recv_len) if recv_len > 0 => {  
5.             // copy to user buffer  
6.         }  
7.         ...  
8.     }  
9. }
```

发送方向则通过 `send_slice()` 将用户数据写入 `smoltcp` 的发送缓冲区，再调用 `net_poll()` 推进协议栈并完成实际发送。

UDP 使用类似的封装方式，同时提供 `bind()`、`connect()`、`sendto()`、`recvfrom()` 等操作，并维护远端地址、本地端口及收发等待队列。

此外，内核实现了部分 `Netlink` 接口，用于兼容 `Linux` 网络配置工具。当前 `StandardNetlinkSocket` 可以处理部分 `RTM_GETLINK`、`RTM_GETADDR` 和 `RTM_NEWADDR` 请求，其中地址修改最终通过 `NET_IFACE.update_ip_addrs()` 更新 `smoltcp` 接口配置。

第 8 章 AI 使用声明

工具：Deepseek v4 pro/flash（posix标准文献查找与翻译），
GPT5.5/5.6(部分代码生成)

使用过程举例：

1. 快速定位测例源码：

- 1) “对于cyclictest_testcode.sh中的程序，指定程序是由哪个文件里的源码编译而来，其运行指令的外部参数对应的执行流是源码的哪些？对应musl/glibc库函数在musl/glibc的哪个文件里？使用了哪些系统调用
- 2) Linux/POSIX标准的快速查询：“140号系统调用的效果是什么？其不同选项对应的效果是什么？”

2. 代码生成：

- 1) “我在tcb里新加了两个字段sched_police以及sched_priority，对于140号系统调用set_priority，先进行参数检查，然后设置优先级”
- 2) “帮我写:如果发现偏移未对齐，则自动向上取整并把中间这些按2M或标准页放进recycled。同时，分配标准页时优先从回收2M页中取并且拆分成若干标准页。”
- 3) “现在的页缓存替换算法实际上只扫描了物理块号最低的4096个页面，这会导致回写不均匀。请你添加一个游标，每次替换时更新一次游标。”

3. 提供建议：

- 1) “对于用户注册的信号处理，似乎用户态要进行上下文的修改，这样的操作是否安全？既然如此，我是否应该去用户态地址空间读取新的上下文？”

AI错误与发现举例：

在初步实现阻塞队列时用LLM生成了部分代码，但在进行阻塞切换前，没有释放阻塞队列的锁，导致在futex相关的测试用例中发生死锁。

发现过程：我们调试的过程是先使用日志快速定位卡死的系统调用，然后再确定断点位置，使用gdb调试。当时卡死在了futex这个系统调用，最后调试时发现在futex唤醒时卡在了获取锁的位置。

我们声明：LLM在我们的项目中被用于重复工作的替代。特别地，对于代码生成，在理解的基础上，我们可能使用AI编写部分系统调用/新功能的早期框架，或作为我们自身想法的快速实现工具。此外，本文档正文全部由人工写成，AI仅被用于项目结构图等少数图表绘制或美化，且经人工复核。

附，使用AI生成的代码内容如下：

- System V IPC msg/shm的syscall封装，File trait实现，以及ds/nsproxy等结构体定义；
- 页缓存管理器部分算法具体实现；
- pagefault的部分代码；
- ext4fs的部分内容，例如getdents()的完善；
- StandardNetlinkSocket rawsocket代码的基础生成；
- 收发报文格式的定义，如IfAddressMsg；
- 部分简单基础系统调用，如sys_sysinfo获得基础信息之类的调用；
- epoll机制的相关早期实现。

此外还有：

- 部分代码片段，算法经我们规划描述后借助AI落地，例如可变页大小分配逻辑和线上决赛阶段部分新功能/优化的快速落地，由AI实现初版后由人工审查验证与修改优化；
- 遇到问题时，用AI在某个功能的路径上快速插入调试桩便于定位问题；

- 部分多处出现的内容有改动（如直接panic的translated_*()访问用户空间 -> 带错误返回的try_transtated_*()）后，写示例让AI批量修改调用模式；
- 部分syscall的参数常量定义；
- 部分syscall的基本逻辑实现。

上述所有用AI生成的代码均经严格的人工审查！

第 9 章 总结与展望

相较于初赛与rCore，我们的内核的健壮性和性能都已有了明显提高，并且对于Linux语义的兼容性也在不断完善，已经能够较为高效地运行起相当部分的Linux程序。

不过，仍有以下不足：

- 部分管理效率低于Linux，系统调用支持数也还不够多；
- 不支持设备树等动态硬件初始化方式；
- 图形化支持尚未实现，只支持串口终端输入输出；
-

下一步，我们将进一步学习Linux或其他前沿研究/成熟工程的实现，继续优化我们的内核，补齐上述短板；特别地，我们准备实现图形栈，在内核上跑起Wayland桌面应用。

参考资料

1. [rCore](https://learningos.cn/rCore-Tutorial-Guide-2025S/) (https://learningos.cn/rCore-Tutorial-Guide-2025S/)
2. [rCore-loongarch](https://github.com/Godones/rCoreloongArch) (https://github.com/Godones/rCoreloongArch)
3. [SubsToKernel](https://gitlab.eduxiji.net/educg-group-36002-2710490/T202510008995695-2720) (https://gitlab.eduxiji.net/educg-group-36002-2710490/T202510008995695-2720)
4. [RocketOS](https://gitlab.eduxiji.net/educg-group-36002-2710490/T202510008995695-2720) (https://gitlab.eduxiji.net/educg-group-36002-2710490/T202510008995695-2720)
5. Linux 源码